

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

REPORT

Laboratory work No. 8

System Calls and Inter-Process Communication in xv6

Rotaru Anastasia, FAF-242

Course: **Operating Systems**

Checked by: Patricia Reitman, asist. univ.

Chişinău, 2026

Contents

1 Introduction **2**

2 Objectives **2**

3 Task 1: Process Execution inside xv6 **2**
3.1 Source code 2
3.2 Execution output 3
3.3 Observations 3

4 Task 2: Bidirectional IPC (Ping-Pong) **4**
4.1 Source code 4
4.2 Execution output 4
4.3 Observations 5

5 Conclusions **5**

1 Introduction

The aim of this laboratory work is to run the process system calls inside xv6, a small educational operating system for RISC-V, and to build a bidirectional communication between two processes with pipes. The first task repeats the fork, exec and wait sequence of the previous laboratory work inside xv6. The second task implements a ping-pong exchange of one byte between a parent and a child process.

2 Objectives

The objectives of this laboratory work are the following. The first objective is to run a program inside the xv6 operating system that creates a child process with `fork()`, replaces the child with another program with `exec()`, and waits for it with `wait()`. The second objective is to compare these calls with the corresponding calls used on Linux in the previous laboratory work, and to explain the meaning of the exit status. The third objective is to build a bidirectional exchange of one byte between a parent and a child process, using two pipes, and to explain why two pipes are needed and which call makes the parent wait for the child.

3 Task 1: Process Execution inside xv6

The program `user/run.c` performs the three steps of the shell for the command `echo hello from the child`. The process creates a child with `fork()`, the child becomes `echo` with `exec()`, and the parent waits for the child with `wait()`. The program uses only the system calls of the xv6 kernel, because xv6 has no C library based on glibc.

3.1 Source code

```
#include "kernel/types.h"
#include "user/user.h"

// the three steps of the shell, inside xv6: echo hello from the child
int
main(int argc, char *argv[]){
    printf("parent: my pid is %d\n", getpid());
    int pid = fork();                    // step 1: copy myself
    if(pid < 0){
        printf("fork failed\n");
        exit(1);
    }
    if(pid == 0){                        // only the child runs this
        printf("child: my pid is %d\n", getpid());
        char *args[] = { "echo", "hello", "from", "the", "child", 0 };
        exec("echo", args);              // step 2: become echo
```

```
17          printf("exec failed\n");          // runs only if exec
    failed
18          exit(1);
19      }
20      int status;
21      wait(&status);                        // step 3: wait for the
    child
22      printf("parent: child %d ended with status %d\n", pid, status
    );
23      exit(0);
24  }
```

Listing 1: Source code of user/run.c

3.2 Execution output

The output below is the expected result from the laboratory handout. Replace it with the output of your own run.

```
$ run
parent: my pid is 3
child: my pid is 4
hello from the child
parent: child 4 ended with status 0
$
```

Listing 2: Output of run inside xv6

The variant that runs `ls` changes the argument array so that it contains only the name `ls` and the terminating zero, and changes the call to `exec` so that it runs `ls`. The variant with a nonexistent program calls `exec` with the name `nosuchprog`. The expected end of its output is the following.

```
exec failed
parent: child 8 ended with status 1
```

Listing 3: Output of run with a nonexistent program

3.3 Observations

Question 1. Which calls does `run.c` use in xv6, and which did the Linux version use? The xv6 program uses `fork()`, `exec()` and `wait()`. The Linux program of the previous laboratory work used `fork()`, `execlp()` and `waitpid()`. The three operations are the same: the process copies itself, the copy is replaced by another program, and the original process waits for the copy to end. The differences are only in the library wrappers. The xv6 `exec` takes the program name and the argument array, and `wait` takes a pointer to a status variable and does not take a process identifier.

Question 2. Who prints the text "hello from the child": the code of `run`, or `echo`? The text is printed by `echo`. The code of `run` only prepares the argument array and calls `exec()`. After the successful call, the child runs the code of `echo`, which writes its arguments to standard output.

Question 3. Why is the status 1 in xv6 and 127 on Linux? The status is decided by the argument of the `exit()` call that follows the failed `exec`. In the xv6 program this

call is `exit(1)`, so the child reports the value 1. In the Linux program of the previous laboratory work the call was `exit(127)`, which is the conventional code for a missing command. The number is chosen by the programmer, not by the operating system.

4 Task 2: Bidirectional IPC (Ping-Pong)

The program `user/pingpong.c` uses two pipes. The first pipe carries one byte from the parent to the child, and the second pipe carries one byte from the child back to the parent. The child prints that it has received the ping before it sends the byte back, and the parent prints that it has received the pong after it reads the byte.

4.1 Source code

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
    int p2c[2], c2p[2];                 // parent-to-child, child-to-parent
    char b;

    pipe(p2c);
    pipe(c2p);

    if(fork() == 0){
        close(p2c[1]); close(c2p[0]);   // the ends the child does not use
        read(p2c[0], &b, 1);
        printf("%d: received ping\n", getpid());
        write(c2p[1], &b, 1);
        exit(0);
    }

    close(p2c[0]); close(c2p[1]);       // the ends the parent does not use
    write(p2c[1], &b, 1);
    read(c2p[0], &b, 1);
    printf("%d: received pong\n", getpid());
    wait(0);
    exit(0);
}
```

Listing 4: Source code of user/pingpong.c

4.2 Execution output

The output below is the expected result from the laboratory handout. Replace it with the output of your own run.

```
$ pingpong
10: received ping
9: received pong
$
```

Listing 5: Output of pingpong inside xv6

4.3 Observations

Question 1. Why are two pipes needed, and not one for both directions? A pipe in xv6 and in Linux is unidirectional: data written to the write end can be read only from the read end. If both processes used one pipe, a process could read back the byte it had just written itself, or two readers could take the byte from each other. Two pipes give each direction its own channel, so each process reads only the data sent to it.
Question 2. Nothing in the code says to wait until the child has replied. Which call makes the parent wait? The parent waits in the call `read(c2p[0], &b, 1)`. This call blocks until the child writes its byte or closes the write end of the second pipe. The final call `wait(0)` then waits until the child has exited.

5 Conclusions

The laboratory work showed that the process system calls of xv6 have the same meaning as those of Linux. The sequence `fork()`, `exec()` and `wait()` runs a program in a child process, and the exit status is chosen by the program that calls `exit()`. The pingpong program showed that two pipes are needed for a two-way exchange, and that a `read()` on an empty pipe blocks the reader until the other process writes. These mechanisms are the same ones used by the shell in the previous laboratory work.